Payment Service Gateway Timeout

Meaning

payment-service calls the external payment-gateway-svc to authorize each transaction. When the gateway's response exceeds payment-service's configured authorization timeout, payment-service marks the transaction as failed/declined locally — even though the charge may have already succeeded on the gateway side — which fires payment-service-error-rate-high. This is a payment-service-local symptom driven by an external authorization dependency, not a database or infrastructure-level cause.

Impact

Users see “payment declined” on a subset of transactions, some of which were actually charged upstream at the gateway, creating a reconciliation gap between payment-service's local record and the gateway's ledger. Only the payment-confirmation step is affected; other services in the purchase flow are not directly impacted, but the reconciliation service will see a spike in unmatched-transaction records within the following billing cycle if the condition isn't caught quickly.

Playbook

1. Confirm which alert fired for payment-service: payment-service-error-rate-high or payment-service-gateway-timeout — check the alert/PagerDuty history.

2. Check payment-service's own resource dashboards (DB connections, CPU, memory) to rule out a local resource cause before assuming a gateway issue.

3. Check the payment-gateway-svc response-time panel: Grafana panel payment-service > Gateway > p99_response_time_ms — confirm whether it spikes at the same time the alert fired.

4. Check payment-service logs for the gateway-timeout error string: grep "gateway authorization timed out" <log-source> and note the first-occurrence timestamp.

5. Check the recent deploy timeline for payment-service and for the gateway client SDK (informational only): GET /repos/org/payment-service/deployments?environment=production — note whether any change touches gateway_timeout_ms or the SDK version.

6. Check the payment-gateway-svc provider's public status page for an active incident on their side.

7. If a provider-side incident is confirmed, no local mitigation exists — monitor and communicate status; do not enable aggressive client-side retries, since retries against a degraded gateway risk duplicate authorizations and compound the reconciliation gap from the Impact section above.

8. If no provider incident is found and gateway latency looks normal, re-check step 2 — the symptom may in fact be a payment-service-local resource issue presenting with gateway-adjacent error messages rather than a true gateway dependency problem.

Diagnosis

1. If the gateway response-time panel from step 3 shows a correlated spike and payment-service's own resource dashboards (step 2) are normal, this is confirmed as a gateway dependency issue, not a payment-service-local resource problem.

2. If the gateway-timeout error string from step 4 first appears at the same time the gateway response-time spike begins (not before), the gateway is the proximate cause, not a coincidental correlation.

3. If a provider status-page incident (step 6) overlaps the alert window, treat it as confirmed root cause and skip to communication — no further local diagnosis will change the outcome.

4. If no provider incident is found and the gateway panel looks normal but the alert is still firing, escalate to payment-service's own on-call to re-examine step 2 in more depth rather than continuing to treat this as a gateway issue.